

Full Length Article

An automated exploitability analysis method based on state machines

Qiang Ruan^a, Zihang Liu^a, Chunlai Du^{a,*}, Li Ma^{a,*}, Yanhui Guo^{b,*}^a School of Artificial Intelligence and Computer Science, North China University of Technology, Beijing, 100144, China^b Department of Computer Science, University of Illinois Springfield, Springfield, IL, 62703, USA

ARTICLE INFO

Keywords:

Automated exploit generation
Exploitability assessment
Exploit state machine
Verifiable exploit primitives

ABSTRACT

Automated Exploit Generation (AEG) is a key technology for addressing the security challenges posed by massive software vulnerabilities. However, current AEG systems suffer from rigid path exploration strategies and binary assessment results, making it difficult to effectively handle complex vulnerability scenarios. To address these challenges, we propose Exploitability Assessment Generation (EAG) model, an automated exploitability analysis method based on state machines. The proposed method departs from the traditional approach of regarding final payload generation as its sole goal, and instead innovatively formalizes the exploitation process as a state-transition process driven by an Exploit State Machine (ESM) and decomposes complex attack behaviors into standardized Verifiable Exploit Primitives (VEPs). By systematically guiding the combined verification of VEPs through the ESM, EAG is able to explore multiple possibilities of vulnerability exploitation, provide validated exploitation paths as supporting evidence for the exploitability assessment and eventually generate Structured and Verifiable Exploit Action Attributes (SVEAA). Experimental results on multiple CTF (Capture The Flag) Pwn challenge datasets and two historical vulnerability cases show that, compared to existing mainstream tools, EAG demonstrates advantages in both accuracy and efficiency for judging exploitability. In the future, we will explore the application of this method in more complex vulnerability environments and further investigate how to incorporate deep learning techniques into our method to enhance adaptability to unknown vulnerability patterns.

1. Introduction

Software vulnerabilities are growing rapidly as the complexity of software systems is increasing day by day. The annual reporting of Common Vulnerabilities and Exposures (CVEs) in 2025 reached a record-breaking number of >48,000 according to the latest data from the National Vulnerability Database (NVD) and were significantly increased compared to previous years (NIST National Vulnerability Database 2026). NIST further reported that CVE submissions had increased by over 260% between 2020 and 2025, with submissions during the first three months of 2026 being nearly one-third higher than those during the same period in 2025 (National Institute of Standards and Technology 2026).

Security teams in enterprises and organizations have serious practical problems, especially in the face of increasing numbers of vulnerabilities. A survey shows that 63% of cybersecurity professionals are unable to perform detailed manual analysis and timely remediation of the large number of vulnerabilities (Koscinski et al., 2025). This

situation demonstrates the inadequacy of the traditional manual-dependent approach to vulnerability management in real-world scenarios and highlights the pressing need for efficient automated solutions that can help security teams in decision-making.

Automated Exploit Generation (AEG) is an important technology in cyber security. It facilitates automated assessment of the exploitability of software vulnerabilities and predicts their potential impact, drastically changing the offensive and defensive landscape (Avgerinos et al., 2014). This technology combines the two fundamental aspects of vulnerability analysis: fuzzing that discovers program irregularities by means of random input and symbolic execution that explores code paths with the help of logical constraint solving. Recent years have seen notable advances in AEG from both academia and industry, resulting in several prototype systems, such as Revery (Wang et al., 2018), BofAEG (Xu and Wang, 2022), and SAEG (Wu et al., 2024), demonstrating capabilities in automated exploit generation.

At the same time, the need for vulnerability assessment has significantly grown. Due to the number of vulnerability reports, it is important

* Corresponding authors.

E-mail addresses: sunkima@mail.ncut.edu.cn (Q. Ruan), lzh2023319020105@mail.ncut.edu.cn (Z. Liu), duchunlai@ncut.edu.cn (C. Du), mali@ncut.edu.cn (L. Ma), yguo56@uis.edu (Y. Guo).

<https://doi.org/10.1016/j.cose.2026.105140>

Received 8 May 2026; Received in revised form 3 August 2026; Accepted 31 August 2026

Available online 4 September 2026

0167-4048/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

for defenders to identify the vulnerabilities that are real threats and prioritize efforts for remediation (Jiang et al., 2025). Reports from industry institutions such as SANS Institute indicate that the lack of practical vulnerability exploitability assessment is a key reason for the occurrence of security incidents (SANS Institute 2026).

Capture The Flag (CTF) competitions, particularly Pwn challenges, serve as a vital platform for testing and advancing vulnerability-exploitation techniques. These challenges require participants to analyze given binary programs within a limited time frame and to find and exploit security vulnerabilities to gain control of the server. In recent years, the design of Pwn challenges has become increasingly intricate, often involving multi-stage exploitation chains and sophisticated memory layouts. This not only places extremely high demands on manual analysis but also reveals the common limitations of existing AEG tools. Specifically, current AEG technology faces two core challenges: On the one hand, the strategies for exploring exploit paths are often rigid. When confronted with vulnerabilities that require complex states or multi-stage interactions to trigger, the path exploration strategies of existing AEG tools tend to be inflexible (Nagy and Hicks, 2019). This rigidity is particularly evident in handling non-control-data attacks, which have become a realistic threat paradigm (Lei and Belaton, 2025). On the other hand, most AEG tools ultimately provide only a success or failure conclusion. In contrast, security analysts need to understand the complete logic, key steps, and prerequisites of the exploit, rather than just a simple binary outcome (Liu et al., 2025).

To address the aforementioned challenges, we propose Exploitability Assessment Generation (EAG), an automated state-machine-based method for vulnerability exploitability analysis. EAG completes exploitability assessment through Verifiable Exploit Primitives (VEPs) and the Exploit State Machine (ESM). In EAG, the VEP is a symbolic representation of an exploitation and information-gathering step used for feasibility verification. On this base, EAG formalizes vulnerability exploitation as a series of state transition processes. It can not only explore more complex exploitation paths but, more importantly, also produce the action attributes as its final output. The action attributes detail the complete path from vulnerability triggering to successful exploitation, the constraints that must be satisfied, and the verification information for each intermediate state. We have implemented a corresponding prototype system named Exploitability Assessment Generation System (EAGS). We evaluated EAG on 25 CTF pwn binaries and two stack-buffer-overflow cases corresponding to CVE-2004-0548 and CVE-2017-9430. The experimental results demonstrate that EAG outperforms existing AEG tools in both the success rate of exploitability judgment and the comprehensiveness of action attributes. The main contributions of this paper are summarized as follows:

- (1) We proposed verifiable exploit primitives and an ESM, and then introduce them into AEG to enhance the analysis capability for multi-layer exploitation paths in complex vulnerability scenarios.
- (2) We proposed EAG, a vulnerability exploitability assessment model, based on the ESM. It not only enables multi-stage vulnerability exploitation, but also compensates for the lack of detail in the results of traditional AEG models, significantly improving exploitability assessment capabilities for complex vulnerability scenarios.
- (3) We developed the prototype system EAGS. By conducting comparative experiments with other AEG models on the same classic CTF dataset and two CVE cases, the experimental results demonstrate that EAG is more effective and superior.

The rest of the paper is organized as follows. Section 2 introduces related work. Section 3 presents the proposed EAG model. Section 4 shows the experiments and results. Section 5 is the conclusion.

2. Related work

Early research on Automated Exploit Generation (AEG) focused on two core techniques: symbolic execution and fuzzing. The pioneering work represented by AEG (Avgerinos et al., 2014) successfully applied symbolic execution to automatically discover and exploit control-flow hijacking vulnerabilities, demonstrating the feasibility of this approach. Subsequent research has continuously deepened this direction by introducing more efficient constraint solvers and path exploration heuristics, and by combining them with techniques for automatically constructing code-reuse attack chains (Xu and Wang, 2022; Vishnyakov and Nurmukhametov, 2021), significantly enhancing the coverage and success rate of AEG systems. However, these methods, which rely on precise program analysis, are generally constrained by their inherent limitations. Among these limitations, the path explosion problem is a significant obstacle to symbolic execution in large and complex software, resulting in low analysis efficiency.

Furthermore, most traditional AEG systems were designed to generate a complete, directly executable exploit, which makes their decision logic relatively inflexible and often yields only binary success/failure conclusions. Although the Common Vulnerability Scoring System (CVSS) score serves as a starting point for assessing vulnerability severity, the scoring results are prone to subjective interpretation and fail to provide critical context regarding exploitability. A vulnerability rated as "Critical" might pose little practical threat due to stringent exploitation conditions or unreachability in specific project deployments. Conversely, a "Medium" severity vulnerability in a critical business component that is easily exploitable could lead to disastrous consequences (Birney, 2026). Transfer learning (Yin et al., 2020) and a real-time concept-adaptive algorithm (Yin et al., 2022) were applied to textual vulnerability descriptions to predict exploitability and address temporal distribution shifts in the data. These methods provide scalable, data-driven estimates at the vulnerability-record level. In contrast, EAG addresses a complementary question at the binary-program level: rather than predicting exploit feasibility from historical vulnerability records, it seeks program-level evidence concerning the feasibility or failure of supported exploitation steps.

In recent years, artificial intelligence (AI) technologies, especially Large Language Models (LLMs) and Deep Neural Networks (DNNs), have introduced new perspectives for automated security analysis (Yang et al., 2024). A great breakthrough is the demonstrated potential of LLMs in vulnerability exploitation, owing to their powerful capabilities in code comprehension, logical reasoning, and code generation. In CTF competitions that demand complex strategies, LLMs such as GPT-4o have already automated solutions to various challenges, sometimes outperforming most human participants (Shao et al., 2024). Meanwhile, LLM agents have been combined with classical planning to coordinate multi-step automated penetration-testing actions (Wang et al., 2025). This demonstrates the feasibility and effectiveness of using LLMs for automated exploit validation. However, although LLMs demonstrate strong capabilities on macro-level reasoning, they remain less reliable when handling low-level binary details, such as memory layouts, address calculations, and path constraints (Jin et al., 2025; Rababah et al., 2024). By contrast, this study focuses on formal and rigorous methods for low-level verification, which is the significance of this research.

Inspired by the aforementioned approaches, we propose EAG, a state machine-based automated method for analyzing vulnerability exploitability. Compared with AEG, it operates at a different level of verification granularity. For selected VEPs, the satisfiability of local exploit-feasibility constraints is used as the condition for an ESM transition, rather than requiring full symbolic emulation of the entire multi-stage interaction. By this design, EAG reduces the growth of symbolic-states caused by end-to-end interaction emulation and preserves intermediate verification evidence in Structured and Verifiable Exploit Action Attributes (SVEAA). EAG abandons the attempt to generate a final

exploit payload in a single step. Instead, it employs a state machine for macro-level, strategic path exploration. At the same time, each critical step is underpinned by verifiable exploit primitives based on symbolic execution to ensure low-level precision. Although EAG itself is not a fuzzer, its design philosophy coincides with neuro-symbolic execution. This approach improves the rigidity of traditional AEGs and compensates for the limitations of LLMs in handling underlying binary details, thereby providing security analysts with deeper, more practical decision-making support.

3. Proposed EAG model

3.1. Overview

We propose EAG, an automated exploitability analysis model for vulnerabilities based on state machines. Its core objective is not merely to achieve automated exploit execution, but also to meticulously model complex exploitation processes, simulate their execution, validate outcomes, and ultimately generate highly structured, comprehensive, and verifiable exploitation action attributes. The action attributes provide security researchers with insights into the entire vulnerability exploitation workflow. The EAG model is shown in Fig. 1.

First, the concept of VEP is introduced. On this base, complex attack behaviors can be decomposed into standardized, independently verifiable units. Subsequently, an ESM is constructed, formalizing vulnerability exploitation as a series of state transition processes. This enables precise description and driving of the entire exploitation attempt process. The state machine combines iterative deepening and selective backtracking for state space search, covering core processing flows for exploitation attempts and leak detection. Crucially, EAG ensures detailed recording and analysis of each VEP's execution outcome, generating comprehensive action attributes of the exploitation process. This eliminates the current limitation in AEG tools that are limited to only binary success/failure conclusions. By preserving the verified intermediate steps and the required conditions as SVEAA, ESM can report partial exploitability evidence even when a complete runnable exploit is not produced.

3.2. Verifiable exploit primitive

We model fundamental exploitation techniques and information-gathering methods as Verifiable Exploit Primitives (VEPs). A VEP is a symbolic specification of an individual exploitation or information-gathering step. It abstracts the preconditions, attacker-controlled symbolic variables, VEP-specific constraints, and intended state effect of that step. It is used to determine whether the corresponding exploitation step is feasible under the current program state and the constraints encoded in the analysis model.

VEP is not itself a vulnerability-specific exploit payload but rather a reusable symbolic representation that instantiates target-specific

candidate structures and verification constraints from the current state. To support this new exploitation concept, we define its identifier, execution and constraint-verification logic, state updates, and SVEAA evidence fields, and then registers it in V_{set} . A VEP is rigorously defined as a triple: $V = (NAME, P_V, E_V)$.

NAME: The unique identifier of the VEP.

Precondition Evaluation (P_V): $P_V(s) \rightarrow \{\text{true}, \text{false}\}$. This is a Boolean function that takes the current state s (see Section 3.3) and determines whether the preconditions for executing this VEP are satisfied.

Execution Core (E_V): $E_V(s) \rightarrow (s', \Lambda)$. This is the core execution unit of the VEP, which receives the current state s and attempts to execute the operation defined by the primitive. Upon completion, it returns a new system state s' and the action attributes Λ . This information serves as the core basis for determining exploitability, verifiably documenting the details of this VEP execution, path constraints, and critical impacts on program state (such as changes in PC values, register states, etc.).

To guide ESM in performing efficient state-space searches, we categorize VEP into two types.

Information Leakage VEPs (V_{leak}): Such primitives are used to gather critical information about runtime state of the target program to enrich the knowledge base K . They form the foundation for achieving the final exploitation by mitigating protection mechanisms and creating execution conditions for subsequent Exploitation VEPs.

Exploitation VEPs (V_{exp}): Such primitives are designed to achieve the ultimate exploitation objective directly. Their successful execution typically signifies the conclusion of the exploitation process.

3.2.1. Information leakage VEPs

The main purpose of information leakage VEPs is to obtain essential information about the runtime state of the target program, thus enriching the knowledge base K . In the EAG model, the process of bypassing current binary protection mechanisms such as Address Space Layout Randomization (ASLR) is abstracted as the dynamic acquisition of missing knowledge through the execution of specific V_{leak} . These primitives serve as prerequisites that enable the subsequent execution of verifiable exploitation primitives.

For example, a typical attack scenario is that attacker can leak the base address of the libc library to bypass ASLR protection by exploiting the lazy binding mechanism of dynamically linked programs. The core of the lazy binding mechanism lies in the differing execution paths between a function's first call and subsequent calls, as illustrated in Fig. 2.

When a call is made for the first time to an external library function indicated by step 1, such as 'puts', the execution flow enters the dynamic linker via its entry 'puts@plt' in the Procedure Linkage Table (PLT) shown as steps 2-4. The dynamic linker is responsible for resolving the actual runtime address of the called function and backfilling this address to its corresponding entry in the Global Offset Table (GOT) shown as the steps 5 and 6. Because any call to this function will subsequently cause the execution flow to jump directly from PLT to the resolved address in GOT['puts'] shown as the steps 8-10, this call will directly enter the 'puts'

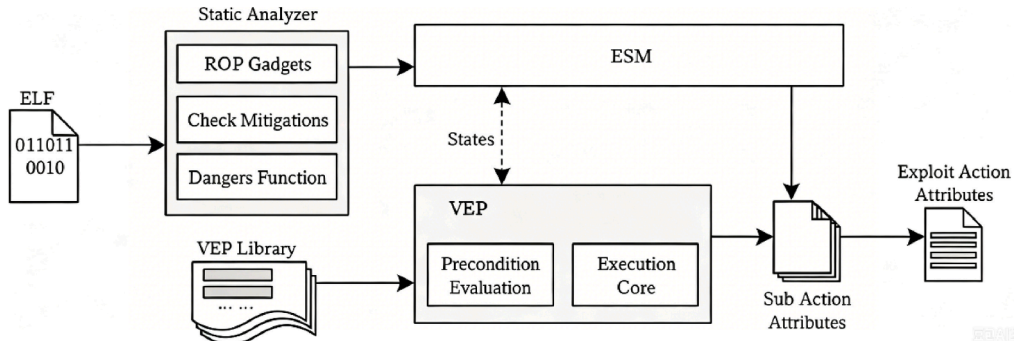
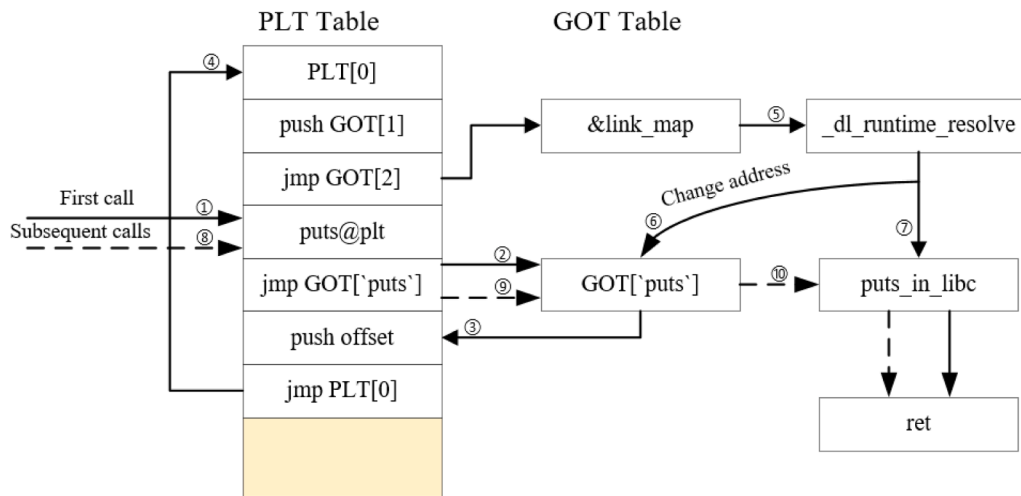


Fig. 1. EAG model.



Solid lines represent the initial resolution path (steps 1-7), while dashed lines illustrate subsequent execution (steps 8-10).

Fig. 2. Lazy binding.

Solid lines represent the initial resolution path (steps 1-7), while dashed lines illustrate subsequent execution (steps 8-10).

within libc library as shown by the solid 7.

In this mechanism, the entry address of the GOT['puts'] is fixed in the data segment of the program. But after the first call to 'puts', the value of the entry changes to the randomized address of the 'puts' in libc. By exploiting this mechanism, an attacker can overwrite the function return address with the address of 'puts@plt' and pass the address of GOT ['puts'] as an argument to the 'puts' function, thereby revealing the base address of libc. This base address is then updated in the knowledge base K, providing the necessary prerequisites for subsequent exploit-based VEPs. Moreover, the process of the information leak is recorded as a significant finding in the final action attributes.

In the EAG model, we proactively explore whether there are paths that could cause out-of-bounds reads and leak critical data from the stack by hooking standard I/O functions and symbolizing relevant parameters, thereby achieving enhanced information-gathering capabilities (Tables 1-5).

3.2.2. Exploitation VEPs

Unlike information-gathering leakage primitives, the verifiable exploitation primitives aim to directly hijack the program control flow, thus enabling formal verification and documentation of attack capabilities. A core feature of the EAG model is that it does not execute a complete attack chain to obtain a shell. Instead, it provides formal exploitability confirmation via program path analysis and state-space exploration. The success metric for the execution core E_V of an exploitative VEP is marked by guiding the program state s to a specific state s' that can unambiguously signify successful exploitation, and recording the feasibility evidence of this state transition in the action attributes Λ .

The general execution process of E_V is designed to consist of the

Table 1
List of Leakage/Mitigation-type VEPs currently supported by EAG.

Name	Condition
Puts Leak	(1) The puts parameter is controllable (2) The address to be leaked is known (3) The leaked data not contain \x00
Stack Pivot	(1) ROP chain length exceeds buffer overflow length (2) A writable memory region exists (3) A leave; ret gadget exists

Table 2

List of exploitation VEPs currently supported by EAG.

Name	Condition
Ret to libc	(1) The program is dynamically linked (2) The libc base address is known (3) The necessary gadgets are present
Ret to win	(1) The program must contain a win function (2) The win function calls a backdoor function
Ret to shellcode	(1) NX protection disabled (2) Presence of jmp rsp instruction or controllable .bss segment
Ret to dlresolve	(1) Partial RELRO or No RELRO (2) Existence of controllable regions for forging data structures and strings

Table 3

Example of action attributes for Ret2Libc case.

EXPLOIT ACTION ATTRIBUTES			
Exploit Summary & Strategy			
Vulnerability Type:		Stack Buffer Overflow	
Exploit Strategy:		Return-to-Libc(ret2libc)	
Precondition Assessment			
✓	Canary is DISABLED		
✓	libc is available		
✓	dynamic is available		
✓	controlled memory length: Required: 31, Available: 978		
✓	Found system@0 × 7ffff7e20d70 and '/bin/sh'@0 × 7ffff7fa8678		
Key Memory Layout			
Type	Address	Size	Permission
libc_base	0 × 7ffff7dd0000	0 × 200,000	r-x
gadget_pop_rdi	0 × 7ffff7dfa3e5	0 × 2	r-x
text_segment	0 × 0(PIE DISABLED)	0 × 2b2	r-x
Exploitation Path Construction			
Step	Action	Method	Parameters
1	prepare_rop_chain	one gadget	gadget_addr
2	verify_exp_feasibility	constraint_solving	logical_payload_size: 32 bytes controlled_size: 978 bytes
3	hijack_control_flow	rop_chain	stack_alignment
4	execute_system	libc_function_call	function_address argument_address
Exploit Feasibility: ✓ HIGHLY			

Table 4

Evaluated results of CTF Pwn programs.

No.	CTF Event Name	Challenge Name	Protection		Time (s)		
			NX	PIE	Zeratool	SAEG	EAGS
1	Angstrom 2021	tranquil	✓	-	10.56	3.30	2.39
2	Angstrom 2023	gaga0	✓	-	7.72	2.41	1.69
3	NA CTF*	Return	✓	-	7.11	2.21	1.14
4	redpwnCTF 2020	coffer	✓	-	N/A	0.57	0.51
5	DCTF 2021	babybof	✓	-	N/A	2.04	0.87
6	DCTF 2021	hotelrop	✓	✓	N/A	2.08	1.29
7	UMDCTF 2021	jump not working	✓	-	N/A	2.25	1.35
8	TamilCTF 2021	name	✓	-	N/A	1.91	1.29
9	UTCTF 2021	resolve	✓	-	N/A	1.97	1.88
10	WPICTF 2020	dorsia1	✓	✓	N/A	2.87	2.06
11	CSICTF 2020	pwn0 × 1	✓	-	N/A	1.90	1.02
12	Angstrom 2019	aquarium	✓	-	N/A	8.20	7.15
13	Angstrom 2023	gaga1	✓	-	N/A	2.90	1.89
14	UTCTF 2020	bof	✓	-	N/A	2.70	1.71
15	CSAW CTF 2021	password	✓	-	N/A	17.48	13.27
16	Bucket CTF 2021	ret4win	✓	-	N/A	N/A	0.48
17	WSC CTF 2022	Babyret	✓	-	N/A	N/A	9.59
18	LIT CTF 2021	madlibs	✓	-	N/A	N/A	12.19
19	WolvCTF 2025	p0wn3d	✓	-	N/A	N/A	0.37
20	WolvCTF 2025	p0wn3d_2	✓	-	N/A	N/A	0.39
21	WolvCTF 2025	p0wn3d_3	✓	-	N/A	N/A	1.66
22	HackitivityCon CTF	Pancakes	✓	-	N/A	N/A	2.89
23	h & HA CTF*	not a baby rop	✓	-	N/A	N/A	10.06
24	corCTF 2021	Chainblock	✓	-	N/A	N/A	4.04
25	UIUCTF 2023	Chainmail	✓	-	N/A	N/A	3.68

* h & HA CTF stands for hackrocks & HackArmour CTF.

* NA CTF stands for Newark Academy CTF.

Note: N/A indicates that the model timed out or failed to analyze the case.

Table 5

Evaluated results of two CVEs.

CVE	Protection		Tools	Time (s)
	NX	PIE		
CVE-2004-0548	-	✓	EAGS	1.19
	-	✓	Zeratool	N/A
	-	✓	SAEG	N/A
CVE-2017-9430	-	-	EAGS(jmp2shellcode)	28.19
	-	-	EAGS(got_leak_puts)	30.43
	-	-	Zeratool	N/A
	-	-	SAEG	N/A

* The reported time is the average of 20 times.

* The brackets represent two different uses of VEP. More detailed SVEAA information of EAGS for two CVEs outputs can be found in [Appendix A](#) and [B](#).

following stages:

A. Logical Construction and Verification of Symbolic Specification

First, we construct a logical symbolic specification according to the attack paradigm of the selected VEP. This logical template is not the real binary data, but a structure that explicitly defines the various components and their logical order which are required for the attack. A generic logical payload structure is defined as:

$$\text{LogicalPayload} = [\text{Padding}] + [\text{Gadgets}] + [\text{Parameter}] + [\text{Target}] \quad (1)$$

where Padding refers to padding data, Gadgets are code segments required by the VEP, Parameter is the parameters required by the VEP and Target is the target address.

Once this logical structure has been established, the EAG model can calculate the minimum number of bytes that have to be overwritten to hijack the control flow. This required length is then compared to the length of the attacker-controlled buffer overflow determined during the initial vulnerability discovery phase. E_V explicitly records the required length and the available length in the action attributes. The condition $\text{required_bytes} \leq \text{available_bytes}$ is a necessary controllable-space

constraint. In addition, the following conditions must also be satisfied for a candidate VEP to confirm exploitability: (1) $P_V(s)$ must holds, (2) $E_V(s)$ verifies a satisfiable vulnerability-triggering path and produces the required validation evidence, and (3) $G(s, \Lambda)$ confirms the VEP-specific target postcondition.

B. Formal Verification of Control-Flow Hijacking Paths

This stage is the core verifiability of VEP. E_V transforms the expected attack goal into one or more logical judgments or assertions. In the case of an attack aimed at jumping to the target function, the EAG model first establishes a core constraint that the value of the program counter (PC) is equal to the address of the target function.

Subsequently, the program path analysis engine of the EAG model will be responsible for analyzing whether an execution path exists from the program entrance to the vulnerability-triggering point that satisfies the above-mentioned PC constraint. If the analysis engine confirms the existence of such a path, it provides the proof of exploitability for the vulnerability logically, eliminating the need to execute the target code or generate specific attack test samples.

3.3. Exploitability assessment method based on ESM**3.3.1. State transitions of the ESM**

The exploitability assessment of EAG is driven by the ESM which is defined as a quintuple: $(\mathbb{S}, A, s_0, \delta, G)$, where $\mathbb{S}$ denotes the set of all possible program states, A represents the set of operations that can affect state transitions, s_0 is the initial program state, $\delta: \mathbb{S} \times V_{set} \rightarrow \mathbb{S}$ is a transition function that maps a current state and a selected VEP to a successor state, and G is a termination decision function that determines whether the state machine should cease its exploration.

State Space ($\mathbb{S}$): It is used to denote all possible program states. The state $s \in \mathbb{S}$ is a composite structure $s = (C, K)$, where: C represents the core state of the program, including the program counter, registers, stack pointer, memory snapshot, etc. K is a knowledge set that stores a collection of known information about the target program in the form of key-value pairs, such as the libc base address, discovered gadget addresses, and canary values. The knowledge set is dynamically expanded

as information-leaking VEPs are successfully executed.

Action (A): It represents the set of all actions that may affect state transitions. In our model, the action set A represents the collection of all defined VEPs, i.e., $A = V_{set}$. An action involves selecting an applicable VEP from this set for execution.

Initial State (s_0): It is the initial state of the program prior to execution. This typically includes the state when the program has just started, along with any known initial knowledge K_0 .

State Transition Function (δ): This function defines how program states transition. When the program is in state s and chooses to execute a VEP V , it transitions to a new state s' . This transition is implemented by the VEP's execution core E_V i.e., $\delta(s, V) = s'$, where s' is one of the two results returned after E_V executes. It is worth noting that although the action attributes Λ generated during state transitions do not directly become part of the next state, they will serve as the core basis for evaluating the goal condition G .

Target Conditions (G): $G(s', \Lambda) \rightarrow \{true, false\}$. G is a decision function that determines whether to terminate the exploration of the state machine after executing a VEP.

The ESM takes the new state s' and the action attributes Λ generated after VEP execution as input and makes a decision according to [Algorithm 1](#).

[Algorithm 1](#) first examines the current action attributes to determine whether a complete exploitation path has been successfully identified. If this is the case, it immediately terminates the entire exploration process. After the current attempt, if it is found that there are no more applicable VEPs for the newly entered state s' , which indicates that the exploration path has reached a dead end, then the current path is also terminated. If a VEP execution neither succeeds nor traps the state machine in a dead end, the ESM may continue attempting other VEPs or paths.

3.3.2. ESM-driven exploitability assessment method

The core of ESM is a deterministic state-space search algorithm. In order to efficiently explore the vast space of potential exploitation paths, ESM adopts a priority-based exploration strategy based on VEP classification, combined with historical state records to avoid loops.

The fundamental idea of this strategy is to prioritize trying the most direct and efficient attack paths. This means that, for any given program state s , ESM always prioritizes executing all available exploitation primitives. These primitives directly lead to the ultimate goal of the exploitation. Only when all exploit-type primitives cannot be executed due to unmet preconditions does ESM take a step back and attempt to execute information-leakage-type primitives. Once a V_{leak} is successfully executed and updates its state, ESM immediately uses this new knowledge to start a completely new exploration, once again prioritizing V_{exp} .

This prioritization strategy can keep the search process focused on finding the most efficient exploitation paths, trying to avoid excessive resource consumption on unnecessary information gathering. [Algorithm 2](#) clearly illustrates the operational flow.

In [Algorithm 2](#), H is an ordered list of successful execution records. Each element (V, Λ, s_{next}) records the successfully executed VEP, the action attributes produced by that execution, and the resulting state.

Algorithm 1

Determination of target condition G .

Input: New state s' , Action Attributes Λ
Output: Terminate state machine *true*, Continue exploration *false*

```

1 function  $G(s', \Lambda)$ 
2 if  $IsExploitSuccessful(\Lambda)$  is true then
3   return true
4 end if
5  $ApplicableVEPs \leftarrow FindApplicableVEPs(s')$ 
6 if  $IsEmpty(ApplicableVEPs)$  is true then
7   return true
8 end if
9 return false
10 end function
```

Failed attempts are not inserted into H .

UpdateBest replaces Λ_{best} with Λ only when Λ represents a higher verified exploitability level according to the predefined order. If the two results have the same level, the first result discovered is retained.

NewKnowledge determines whether the successor state contains new verified information that is absent from the current state.

Let $s = (C, K)$ and $s_{next} = (C_{next}, K_{next})$. NewKnowledge is defined as $NewKnowledge(s, s_{next}) \Leftrightarrow K_{next} \setminus K \neq \emptyset$.

First, the ESM initializes the current state s with s_0 , creates an empty ordered execution history H , and initializes Λ_{best} . H contains only successful execution records in the form (V, Λ, s_{next}) , whereas Λ_{best} retains the non-terminal result showing the greatest verified progress toward the exploitation goal.

Second, in each exploration round, the ESM first identifies the applicable V_{exp_set} and then executes them in the predefined priority order. If an V_{exp_set} satisfies both $IsSuccess(\Lambda)$ and the goal condition $G(s_{next}, \Lambda)$, its successful execution record is appended to H , and the algorithm immediately returns the final action attributes and execution history. Otherwise, **UpdateBest** determines whether the current action attributes should replace the previously retained result ([Lines 5–16](#)).

If V_{exp_set} cannot reach the goal, the ESM evaluates the applicable V_{leak_set} . A V_{leak_set} causes a state transition only when its execution succeeds and the successor state contains new knowledge. The successful record is appended to H , the current state is updated, and the continue Loop explicitly starts the next exploration round. The V_{exp_set} are then reconsidered using the newly acquired knowledge ([Lines 18–28](#)).

Finally, if no V_{exp_set} reaches the goal and no V_{leak_set} adds new knowledge, the current exploration cannot make further progress. The algorithm2 will terminate and return the best action attributes retained so far, together with the history of successful execution ([Lines 30–32](#)).

Complexity Analysis. Let $N = |V_{exp}| + |V_{leak}|$, and let D be the maximum number of successful information-leakage transitions. Each iteration examines at most N VEPs, and every accepted leakage transition strictly adds new knowledge. Therefore, the outer loop executes at most $D + 1$ times.

Let C_V denote the maximum loop cost of checking, executing, and verifying one VEP. The worst-case time complexity is: $O((D + 1)NC_V)$.

If each VEP operation is treated as unit cost, the ESM scheduling complexity is:

$$O((D + 1)N).$$

The candidate sets require $O(N)$ space, while H stores at most $D + 1$ successful records; thus, excluding the internal workspace of individual VEPs, the space complexity is:

$$O(N + D).$$

Since NewKnowledge requires every accepted transition to strictly extend a finite knowledge set, the algorithm eventually terminates.

Algorithm 2

ESM-driven exploitability assessment.

Input: $ESM = (\mathbb{S}, A, s_0, \delta, G), V_{set}$
Output: Action Attributes Λ_{best}

begin

```

1  $s \leftarrow s_0, H \leftarrow \emptyset$ 
2  $\Lambda_{best} \leftarrow \text{CreateEmptyInformation}()$ 
3  $\blacksquare$ 
4 loop
5  $\text{ApplicableVEPs} \leftarrow \{V \in V_{set} \mid P_V(s) = \text{true}\}$ 
6  $V_{exp\_set} \leftarrow \text{ApplicableVEPs} \cap V_{exp}$ 
7 for each  $V$  in  $V_{exp\_set}$  do
8  $s_{next}, \Lambda \leftarrow E_V(s)$ 
9 if  $G(s_{next}, \Lambda)$  is true and  $\text{IsSuccess}(\Lambda)$  then
10  $H.append((V, \Lambda, s_{next}))$ 
11  $\Lambda_{best} \leftarrow \Lambda$ 
12 return  $(\Lambda_{best}, H)$ 
13 end if
14  $\blacksquare$ 
15  $\Lambda_{best} \leftarrow \text{UpdateBest}(\Lambda_{best}, \Lambda)$ 
16 end for
17  $\blacksquare$ 
18  $V_{leak\_set} \leftarrow \text{ApplicableVEPs} \cap V_{leak}$ 
19 for each  $V$  in  $V_{leak\_set}$  do
20  $s_{next}, \Lambda \leftarrow E_V(s)$ 
21 if  $\text{IsSuccessfulLeak}(\Lambda)$  and
22  $\text{NewKnowledge}(s, s_{next})$  then
23  $H.append((V, \Lambda, s_{next}))$ 
24  $\Lambda_{best} \leftarrow \text{UpdateBest}(\Lambda_{best}, \Lambda)$ 
25  $s \leftarrow s_{next}$ 
26 continue loop
27 end if
28 end for
29  $\blacksquare$ 
30 return  $(\Lambda_{best}, H)$ 
31 end loop
32 end function

```

3.4. Structured and verifiable exploit action attributes (SVEAA)

The action attributes generated in this work adhere to a rigorous and unified data processing procedure, decomposing the complex exploitation process into multiple dimensions for presentation. In contrast to previous output formats that merely provided simple binary success/failure conclusions, the action attributes present significantly richer details. Each step and conclusion within the action attributes is supported by explicit evidence derived from symbolic execution, constraint solving and memory analysis, enabling researchers to trace and verify every stage of the exploitation path.

The structure of the SVEAA consists of five core components:

Exploit Summary & Strategy: It is an overview of the action attributes, clarifying the type of vulnerability being analyzed, the core exploitation strategy ultimately selected and the overall attack objectives.

Precondition Assessment: It is a detailed list of all the preconditions that need to be met for the selected exploitation VEP to execute successfully, and indicates the verification status and evidence of each condition.

Key Memory Layout: It is a representation of all critical memory areas involved in attack process, including memory type, runtime address, size and permissions.

Exploitation Path Construction: This is the core part of SVEAA, which shows the complete path from triggering vulnerability to ultimately hijacking control flow step by step. Each step contains specific action operations, usage methods, and key parameters. This is an ordered presentation of the execution results of each E_V in the successful VEP sequence, which completely recreates the optimal attack path explored by ESM.

Exploit Feasibility: Based on the analysis of all the above dimensions, a definitive exploitability rating will be assigned to the

ultimate exploitability of vulnerabilities. A 'HIGHLY' rating is assigned only when all VEP preconditions are met, the vulnerability-triggering path and VEP-specific constraints are verified, and the logical payload size remains within the controllable space.

3.5. Example

To illustrate the exploration strategy using state machines and the generation process of action attributes, this section provides a detailed explanation of both, which is exemplified by the code presented in Fig. 3.

When the execution flow enters the function 'vul_func', EAG replaces the input to the vulnerable function 'gets()' with a symbolic byte stream. When 'vul_func' finishes its execution and is ready to return, the return address on the stack has been overwritten by this symbolic input stream. At this point, the PC is no longer a single fixed address, but rather a symbolic expression that can be arbitrarily determined by the attacker. This state explicitly indicates that the control flow of the program can be hijacked. This critical event activates the state machine of exploitation analysis. EAG immediately captures the current program state C_0 and combines it with the initial knowledge K_0 obtained from static analysis to form the initial state $s_0 = (C_0, K_0)$ of the ESM. The objective of the ESM is to start from the initial state s_0 and, by verifying a sequence of VEPs, determine whether a feasible VEP sequence exists under the encoded constraints.

After ESM starts, it enters its core loop and begins selecting actions from its action set A . Following a priority-based exploration strategy, ESM prioritizes attempting verifiable exploitation primitives V_{exp} that can directly achieve the goal. ESM first attempts to use the simplest and lowest-cost VEP. If it fails, ESM will start calling function P_V of Ret2Libc. This function checks the preconditions for applying the Ret2Libc VEP, which explicitly requires that the base address of libc must be included

```

1  /*ret2libc_example.c*/
2  void print_banner() {
3      puts("VEP demonstration.");
4      puts("This program contains a vulnerability suitable.");
5  }
6
7  void vul_func() {
8      char buffer[64];
9      puts("Please provide your input to test the Exploit State Machine.");
10     // gets() lacks bounds checking.
11     gets(buffer);
12     printf("Your input was: %s\n", buffer);
13 }
14
15 int main() {
16     setvbuf(stdout, NULL, _IONBF, 0);
17     setvbuf(stdin, NULL, _IONBF, 0);
18
19     print_banner();
20     vul_func();
21
22     puts("Program finished.");
23     return 0;
24 }

```

Fig. 3. Example of Ret2Libc.

in the knowledge base K . In the initial state s_0 , the knowledge base K_0 contains only static analysis information and has not yet obtained any runtime addresses. Therefore, $P_V(s_0)$ returns *false*. Because the preconditions are not met, the Ret2Libc VEP is not applicable in the current state. ESM continues to evaluate other exploitation-type VEPs, but they also fail due to their dependency on libc addresses. At this point, ESM determines that the direct exploitation VEPs cannot yet be verified with the current knowledge set. When all V_{exp} are unavailable, ESM automatically switches to attempting to verify the information-leaking VEP V_{leak} , with the aim of expanding the knowledge set K to create new exploitation conditions.

ESM searched within the V_{leak} set to identify and obtain an exploitation. Before calling function 'vul_func', the function 'print_banner' had already invoked function 'puts()'. By hooking the standard I/O functions, EAG determined that the runtime address of the function 'puts()' had been resolved in the Global Offset Table (GOT). This fully satisfies the preconditions for a Puts Leak. ESM decides to verify this VEP, with its verification core E_V , initiating formal modeling and verification of an information-leak attack. E_V first instantiates a symbolic representation of the leakage step. This path exploits a known overflow vulnerability to construct a symbolic ROP layout for puts(puts@got) by overwriting the return address with the address of puts@plt while passing the address of puts@got as an argument to function 'puts()'. In addition, E_V transforms this attack path into path constraints. It verifies whether a symbolic input assignment exists that precisely achieves the required symbolic stack layout, thereby satisfying all conditions for the puts(puts@got) call. Once confirming the leak path is viable, the symbolic output of puts(puts@got) should be captured. This output represents a symbolic expression representing the runtime address of the function 'puts()' within libc. By querying the built-in libc library signature database, the base address of libc can be calculated by using the known 'puts()' offset. It is worth mentioning that EAG can be configured at this stage to execute in a real environment to obtain the exact address, or it can be purely verified at the logical level and use a symbolized base address for subsequent inference.

The successful execution of the VEP resulted in a state transition: $\delta(s_0, V_{leak}) = s_1$. The new state $s_1 = (C_1, K_1)$ not only contains the

updated core program state but also expands knowledge base K_1 . Simultaneously, E_V generates interim action attributes documenting the complete verification process of this information leak. It marks the exploitability as moderate level, indicating significant progress but not yet the final exploitation. Therefore, ESM enters a new exploration. Based on the updated state s_1 , when VEP of Ret2Libc is re-evaluated, its precondition $P_V(s_1, K_1)$ returns *true* because the knowledge base K_1 now contains the required libc base address. The execution core E_V of Ret2Libc then initiates a process that deeply integrates verification information and action attributes. E_V uses the known base address of libc to compute runtime addresses for the function 'system', the "/bin/sh" string, and key ROP gadgets such as 'pop rdi; ret'. These addresses and memory areas are added to the memory_layout section of the action attributes simultaneously.

E_V constructs the logical structure of the final ROP chain at the symbolic level as shown below.

[gadget_addr, binsh_addr, system_addr]

Subsequently, it adds a core constraint to the constraint solver, which is whether there exists an input that can make the covered return address area on the stack exactly equal to the logical sequence of this ROP chain.

When the constraint solver returns 'satisfiable', this provides feasibility evidence for the Ret2Libc VEP under the encoded constraints. Finally, the judgment function G of ESM receives the action attributes which satisfy the ESM's termination condition. The ESM then halts exploration and outputs this structured information as the final result. The action attributes record the feasibility evidence and provide a rigorous and reproducible logical deduction of the VEP verification process. This clearly presents both the ESM's exploration path and the VEP's verification results.

4. Experiments and results

4.1. Experimental environment

We implemented the EAG prototype system (EAGS) based on the binary analysis platform angr (Shoshitaishvili et al., 2016), the exploit development library pwntools (Gallopsted, 2026), the Z3 constraint solver (de Moura and Bjørner, 2008), and the disassembly framework radare2 (Radareorg, 2026), using Python 3.8 as the programming language. The operating system used was Ubuntu 22.04 64-bit. The evaluation dataset comprised 25 CTF Pwn challenge programs obtained from CTFTime (CTFTime.org, 2026). We compared EAGS with the well-known binary tools Zeratool (Zeratool, 2026) and SAEG. Furthermore, to complete the experiment against real vulnerabilities, we make experiments on some public CVE vulnerabilities: CVE-2004-0548 in Aspell's word list compression function and CVE-2017-9430 in dnstracer*. Each tool was run 20 times on each vulnerable software. For EAGS, a run is counted as completed when it returns structured results with an explicit verification level. For Zeratool, a positive result indicates that an unconstrained state has been found. For SAEG, applying its released implementation to the reproduced CVE builds required additional adaptation of the analysis environment, input interfaces, and angr-dependent components.

* Refer to Appendix A and B for detailed information on CTF competition and two CVEs.

4.2. Experimental results

EAGS specializes in the automated and efficient judgment of exploitability. EAGS maintains high accuracy in judging whether vulnerabilities are exploitable and provides more comprehensive action attributes.

4.3. Result analysis

We analyze some typical experimental cases to demonstrate the effectiveness of EAG.

4.3.1. Case 1 analysis

To validate the enhanced exploitability of the EAGS, we selected a dataflow-oriented vulnerability case **p0wn3d_2** in WolvCTF 2025. This case has a stack buffer overflow vulnerability, but the exploitation method does not involve hijacking control flow. Instead, it altered the execution path by overwriting the values of critical local variables on the stack, thereby triggering the logic of print the flag.

Experimental results show that traditional AEG models generally fail to handle this type of data-flow-based attack. For these models, the PC controlled by the attacker's input serves as both the starting point for analysis and the main target for exploitation judgment. These exploitation generation modules are hard-coded to search for paths that hijack control flow under these specific conditions.

Consequently, when encountering vulnerabilities like the one in this case which does not involve PC hijacking, although the vulnerability does exist, its core exploitation logic can never be triggered, ultimately leading to a failure in the analysis. When handling such non-control flow hijacking vulnerabilities, ESM reframes the problem as follows: given the current state, whether there exists a reachable path that can lead to any predefined successful state. This approach extends the exploitation paradigm from control flow hijacking to program state manipulation. The experimental results show that EAGS was able to successfully identify the state-manipulation exploitation path in this case and judge its exploitability.

This case demonstrates that by extending the vulnerability exploitation analysis paradigm from simple control flow hijacking to program state manipulation, ESM can effectively mitigate the limitations of traditional AEG models when dealing with non-control-flow-oriented attacks.

4.3.2. Case 2 analysis

To further validate EAG's capability and efficiency in handling traditional control-flow hijacking vulnerabilities, we make an analysis on the second case not a baby rop in h&HA CTF. This case represents a classic ret2libc scenario with a stack overflow vulnerability. Although well-established exploitation techniques exist for such vulnerabilities, the experimental results once again demonstrated that the Zeratool and SAEG models have failed to analyze this vulnerability, whereas EAGS successfully generated high-confidence exploitation action attributes.

The failure of traditional AEG models has exposed shortcomings in their analysis methods. These methods rely too heavily on the precise reproduction of interaction sequences. They endeavor to precisely simulate the entire information-leakage process, construct the ROP chain to invoke the function 'puts', deliver a logical payload, and parse the base address of 'libc' from an I/O stream filled with symbolic variables. This process places extremely high demands on the symbolic execution engine and constraint solver. It is highly prone to failure due to overly complex path states or the failure of heuristically parsing output, causing the simulation process to crash and thereby interrupting the exploration of the entire exploitation chain.

EAGS has adopted a more efficient and robust strategy, which verifies the feasibility of exploitation intent rather than precisely simulating the entire exploitation process. When facing with the requirement to leak an address, EAGS does not attempt to simulate and parse the output of the function 'puts'. Instead, its execution core, E_v , transforms the address leakage problem into a constraint satisfiability problem. Once the solver confirms that the path exists, EAGS deems it feasible to leak the base address of libc. Subsequently, ESM updates a symbolic variable representing the base address of libc to its knowledge set K . This avoids complex emulation of I/O.

This case demonstrates the progress of EAG in analytical methods. By

transforming high-cost real-time simulation into low-cost intent verification, the incapability of pure symbolic execution in handling multi-stage stateful attacks is effectively avoided.

4.3.3. Analysis of CVE results

EAGS and vanilla angr both returned positive results for the two CVE cases. The difference lies in the reported information. Vanilla angr indicates that a feasible path was found, whereas EAGS also records the selected VEP, verified prerequisites, relevant memory information, and the constraint result in SVEAA. SAEG and Zeratool did not complete either case because it encountered an angr API compatibility error. These experiments show that EAGS can produce structured exploitability evidence for the two tested CVE vulnerabilities.

4.4. Discussion

The core advantage of EAG is that it no longer pursues the generation of specific attack payloads, but models and verifies the vulnerability exploitation process through VEP and ESM. This idea makes EAG more versatile, especially when handling complex scenarios such as non-control-flow hijacking vulnerabilities, complex path constraints, and symbol state explosions caused by multi-stage interactions. It shows significant robustness in these situations. In addition, with the introduction of structured exploitation action attributes, these attributes not only enhance the interpretability and verifiability of vulnerability analysis in the target program, but also provide a judgment basis used for subsequent manual reviews or security policy development. These evidence-oriented mechanisms could be incorporated into an existing AEG framework, but this would require changing its verification unit, transition semantics, termination criterion, and output model. Regarding this issue, ESM proposes a step-by-step, evidence-based method for analysing the exploit feasibility of vulnerabilities and provides a unified formalization of these changes.

Nevertheless, there are also limitations at present. Although the uniform interface allows new VEPs to be integrated without modifying the ESM, encoding technique-specific semantics still requires manual effort, particularly across different architectures, ABIs, libraries, and mitigations. Unsupported techniques may therefore cause false-negative assessments.

5. Conclusion

We proposed EAG model and implemented prototype system EAGS, an automated analysis method of vulnerability exploitability based on state machines. EAG has moved away from the traditional approach of generating attack payloads, instead focusing on formally modelling and verifying exploitation processes.

To achieve this goal, we introduce two cores: verifiable exploitation primitives and exploitation state machines. By using a state machine-driven search strategy, EAG decomposes complex attack behaviors into a series of standardized, independently verifiable atomic steps. It then systematically explores and verifies potential exploitation paths. It not only extends the judgment of vulnerability exploitation from single control-flow hijacking to a manipulation of program state, but also effectively avoids the incapability of traditional symbolic execution in simulating multi-stage interactions.

Experimental evaluations on the CTF Pwn dataset that covers various types of buffer overflow vulnerabilities and two CVEs demonstrate the significant advantages of EAG. Case analysis further confirms EAG's ability to successfully handle non-control-flow attacks beyond the capabilities of traditional AEG, while providing precise diagnostic action attributes when exploitation paths fail. The action attributes significantly enhance the interpretability of automated analysis results and provide a clear decision-making basis for defense teams. By shifting focus from payload generation to exploitability assessment and action attributes, EAG shows potential to improve the robustness of analysis

and interpretability of results.

Funding statement

This work was supported by the National Key R&D Program of China (Grant No. 2024YFE0200500) and the National Natural Science Foundation of China (Grant No. 62172006).

CRediT authorship contribution statement

Qiang Ruan: Writing – original draft, Visualization, Validation, Software, Methodology. **Zihang Liu:** Writing – original draft,

Visualization, Software, Methodology, Investigation. **Chunlai Du:** Writing – review & editing, Project administration, Methodology, Funding acquisition, Conceptualization. **Li Ma:** Writing – review & editing, Project administration, Conceptualization. **Yanhui Guo:** Validation, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. EAGS SVEAA outputs for CVE

A.1 SVEAA for CVE-2004-0548

[Table A1](#), [A2](#), [A3](#)

Table A1

SVEAA for CVE-2004-0548.

EXPLOIT ACTION ATTRIBUTES— CVE-2004-0548

Exploit Summary & Strategy

Vulnerability Type:

Stack-based Buffer Overflow

Exploit Strategy:

Candidate GOT-based libc disclosure via fputs@PLT

Precondition Assessment

✓

Vulnerable path was reached at analysis-model address $0 \times 666CDEAD000015A4$

✓

A symbolic control object was detected, and complete return-address control was established: 8/8 bytes

✓

The binary is dynamically linked, and fputs@PLT and fputs@GOT are available

✓

A 56-byte candidate fputs GOT-disclosure ROP layout was constructed

✓

The encoded constraints for return-address control, the fputs disclosure template, and return after disclosure are satisfiable

Key Memory Layout

Type	Address	Size	Permission
text_segment	$0 \times 666CDEAD000010E0^*$	0×545	r-x
fputs_plt	$0 \times 666CDEAD00001094^*$	0×10	r-x
fputs_got	$0 \times 666CDEAD00003FB0^*$	0×8	r-
stack	$0 \times 7FFFFFFEF50$	0×2000	rw-
main_function	$0 \times 666CDEAD00001349^*$	0×100	r-x
entry_point	$0 \times 666CDEAD000010E0^*$	0×20	r-x

Exploitation Path Construction

Step	Action	Method	Parameters
1	build_fputs_leak_rop_chain	ROP-chain construction	Leak function: fputs Leak target: fputs@GOT stdout_address: $0 \times 666CC0DE002046A8$ Return target: $0 \times 666CDEAD000010E0$ Payload length: 56 bytes
2	verify_fputs_leak_feasibility	Constraint solving	Controlled return-address bytes: 8/8; constraints: controlled return address, fputs disclosure template, and return after disclosure; solver result: SAT

Exploit Feasibility: ✓ MEDIUM

The candidate GOT-disclosure VEP is locally satisfiable under the encoded symbolic constraints. However, the analysis did not execute the candidate chain in a real process, observe a concrete leaked address, or resolve the libc base.

* Addresses prefixed with $0 \times 666C$ are analysis-model addresses and do not represent concrete runtime addresses.

A.2 SVEAA for CVE-2017-9430

Table A2

SVEAA for CVE-2017-9430 In jmp2shellcode.

EXPLOIT ACTION ATTRIBUTES— CVE-2017-9430

Exploit Summary & Strategy

Vulnerability Type:

Stack-based Buffer Overflow

Exploit Strategy:

Jump-to-shellcode using a jmp esp gadget

Precondition Assessment

✓

The vulnerable path was reached in main.

✓

Complete control of the saved return address was established: 4/4 bytes.

✓

Stack canary and NX are disabled.

✓

An executable jmp esp gadget was identified at 0 × 804E22F.

✓

The combined constraints for return-address control and shellcode injection are satisfiable.

Key Memory Layout

Type	Address	Size	Permission
------	---------	------	------------

(continued on next page)

Table A2 (continued)

EXPLOIT ACTION ATTRIBUTES— CVE-2017-9430				
jmp_esp_gadget		0 × 804E22F	0 × 2	r-x
Candidate shellcode location		0 × 666C3424*	0 × 18	rwX
Exploitation Path Construction				
Step	Action	Method	Parameters	
1	prepare_shellcode	Standard shellcode construction	Type: execve("/bin/sh") Shellcode length: 24 bytes	
2	verify_exploit_feasibility	Constraint solving	jmp esp: 0 × 804E22F; modeled offset to overwritten EIP: 4 bytes; shellcode injection begins at modeled offset 8; candidate payload size: 28 bytes; solver result: SAT	
3	hijack_control_flow	Indirect jump	Overwrite EIP with 0 × 804E22F; execute jmp esp; technique: stack-based buffer overflow	
4	execute_shellcode	Modeled direct execution	Location: stack Analysis address: 0 × 666C3424 Size: 24 bytes Required permission: rwX	
Exploit Feasibility: ✓ HIGHLY The vulnerable path, complete 4-byte return-address control, executable jmp esp gadget, and shellcode-injection constraints were verified under the symbolic model.				

* Addresses prefixed with 0 × 666C are analysis-model addresses and do not represent concrete.

Table A3

SVEAA for CVE-2017-9430 In got_leak_puts.

EXPLOIT ACTION ATTRIBUTES— CVE-2017-9430				
Exploit Summary & Strategy				
Vulnerability	Stack-based Buffer Overflow			
Type:				
Exploit Strategy:	GOT-based libc disclosure via puts@PLT			
Precondition Assessment				
✓	The vulnerable path was reached in main, and a symbolic control object was detected			
✓	The binary is dynamically linked, and puts@PLT and puts@GOT are available			
✓	A 12-byte GOT-disclosure ROP template was constructed			
✓	The GOT-disclosure template constraints are satisfiable			
	Complete return-address control was not established: 1/4 bytes controlled			
Key Memory Layout				
Type	Address	Size	Permission	
text_segment	0 × 8049,540	0 × 39D9	r-x	
puts_plt	0 × 80493E4	0 × 10	r-x	
puts_got	0 × 8050,054	0 × 4	rw-	
main_function	0 × 804C48b	0 × 100	r-x	
entry_point	0 × 8049,540	0 × 20	r-x	
Exploitation Path Construction				
Step	Action	Method	Parameters	
1	build_got_leak_rop_chain	ROP-chain construction	Leak function: puts; leak target: puts@GOT; recorded re-entry target: entry point at 0 × 8049,540; candidate payload length: 12 bytes	
2	verify_got_leak_feasibility	Constraint solving	Template constraints: controlled-return-address expression, GOT-disclosure layout, and return after disclosure; solver result: SAT	
3	verify_payload_injectability	Attacker-control assessment	Required return-address control: 4 bytes; verified control: 1 byte;	
Exploit Feasibility: ✓ MEDIUM The GOT-leak template was locally constraint-satisfiable, but complete attacker control over the required ROP-chain bytes was not established.				

In the CVE-2017-9430 experiment, two different verification strategies emerged. These two reports were produced in different runs, rather than from a single ESM execution. This difference stems from the ESM search mechanism of EAGS: whenever an unconstrained state is found during symbolic execution, ESM will traverse all 14 registered exploitation VEPs and information leak VEPs in a fixed order to check whether the pre-conditions of each primitive are met by the current state. That is to say, this is related to their selection of different VEP primitives. In one case, the constraints of the jmp2shellcode VEP were satisfied, and the result was returned as a high-confidence exploitation strategy. In the other case, jmp2shellcode did not reach the terminal condition, after which EAGS evaluated the GOT-leak VEP and produced the candidate_got_leak_puts report. The latter result shows that the local leakage template was constraint-satisfiable.

Appendix B. Benchmark manifest

This appendix documents the provenance of the benchmark programs used in the evaluation. The public source listed for each challenge refers to an organizer-maintained repository, a public preservation archive, or a participant-maintained mirror from which the challenge identity and artifacts can be verified. Because some original competition websites are no longer available, a preserved public copy is reported where an official repository could not be located (Tables B1 and B2).

B1. Provenance of the CTF Pwn Benchmarks

Table B1

Provenance of the CTF Pwn benchmarks.

No.	Event	Challenge / artifact	Public source	Source type
1	AngstromCTF 2021	tranquil / tranquil	Public challenge archive	Public archive
2	AngstromCTF 2023	gaga0 / gaga0	gaga challenge files	Public archive
3	Newark Academy CTF 2022	Return / ret2	Preserved binary and source	Participant mirror
4	redpwnCTF 2020	coffer-overflow-1 / coffer-overflow-1	Official challenge repository	Official repository
5	DCTF 2021	Baby bof / baby_bof	Public challenge archive	Public archive
6	DCTF 2021	Hotel ROP / hotel_rop	Public challenge archive	Public archive
7	UMDCTF 2021	Jump Not Working / JNW	Public challenge archive	Public archive
8	TamilCTF 2021	Name Server / name-serv	Preserved challenge files	Participant mirror
9	UTCTF 2021	Resolve / resolve	Public challenge archive	Public archive
10	WPICTF 2020	dorsia1	CTFtime challenge record	Challenge record
11	csictf 2020	pwn intended 0 × 1 / pwn-intended-0 × 1	Official challenge repository	Official repository
12	ångstromCTF 2019	Aquarium / aquarium	Public challenge archive	Public archive
13	ångstromCTF 2023	gaga1 / gaga1	gaga challenge files	Public archive
14	UTCTF 2020	bof / pwnable	Public challenge archive	Public archive
15	CSAW CTF 2021 Qualifiers	Password Checker / password_checker	Official challenge repository	Official repository
16	BuckeyeCTF 2021	ret4win / chall	Official challenge repository	Official repository
17	Wolverine Security Conference/CTF 2022	babyRet / ret0	Public challenge archive	Public archive
18	LIT CTF 2021	madlibs / madlibs	Official challenge repository	Official repository
19	WolvCTF 2025	p0wn3d / chal	Official challenge repository	Official repository
20	WolvCTF 2025	p0wn3d_2 / chal	Official challenge repository	Official repository
21	WolvCTF 2025	p0wn3d_3 / chal	Official challenge repository	Official repository
22	H@ctivityCon CTF 2020	Pancakes / pancakes	Public challenge archive	Public archive
23	hackrocks & HackArmour CTF 2022	Not a Baby ROP / not-a-baby-rop	Preserved binary and exploit files	Participant mirror
24	corCTF 2021	Chainblock / chainblock	Public challenge archive	Public archive
25	UIUCTF 2023	Chainmail / chal	Official challenge repository	Official repository

B2. Manifest of the CVE Cases

Table B2

Manifest of the CVE cases.

CVE	Vulnerable Program	Source Release	Trigger Interface
CVE-2004-0548	GNU Aspell 0.50.5 (word-list-compress)	Aspell 0.50.5 source archive	An overlong word-list entry supplied to word-list-compress through standard input
CVE-2017-9430	dnstracer 1.9 (dnstracer)	dnstracer source repository	An overlong domain-name argument supplied through the command line

Data availability

The data that support the findings of this study are available on All about CTF at <https://ctftime.org/>.

References

- Avgerinos, Thanassis, Cha, Sang Kil, Rebert, Alexandre, Schwartz, Edward J., Woo, Maverick, Brumley, David, 2014. Automatic exploit generation. *Commun. ACM* 57 (2), 74–84. <https://doi.org/10.1145/2560217.2560219>.
- Birney, Robin, 2026. Beyond CVSS: project context, exploitability and reachability of vulnerabilities. Available. <https://www.getsafety.com/blog-posts/beyond-cvss-project-context-exploitability-and-reachability-of-vulnerabilities>. Accessed: 2026-08-01.
- CTFtime.org, 2026. All about CTF. Available. <https://ctftime.org/>. Accessed: 2026-08-01.
- de Moura, Leonardo, Björner, Nikolaj, 2008. Z3: An efficient SMT Solver, Tools and Algorithms for the Construction and Analysis of Systems (TACAS), 4963. Springer, Berlin/Heidelberg, Germany, pp. 337–340. https://doi.org/10.1007/978-3-540-78800-3_24.
- Gallopsled, 2026. pwntools. Available. <https://github.com/Gallopsled/pwntools>. Accessed: 2026-08-01.
- Yuning Jiang; Nay Oo; Qiaoran Meng; Hoon Wei Lim; Biplab Sikdar. A survey on vulnerability prioritization: taxonomy, metrics, and research challenges. *arXiv*, 2025, 1–32. [10.48550/arXiv.2502.11070](https://arxiv.org/abs/2502.11070).
- Jin, David, Fu, Qian, Li, Yuekang, 2025. Good news for script kiddies? Evaluating large language models for automated exploit generation. In: Proceedings of the 46th IEEE Symposium on Security and Privacy Workshops (SPW 2025). San Francisco, USA. IEEE Computer Society, pp. 278–282. <https://doi.org/10.1109/SPW67851.2025.00039>.
- Koscinski, Viktoria, Nelson, Mark, Okutan, Ahmet, Falso, Robert, Mirakhorli, Mehdi, 2025. Conflicting scores, confusing signals: an empirical study of vulnerability scoring systems. In: Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS'25). Taipei, Taiwan. ACM, pp. 1905–1918. <https://doi.org/10.1145/3719027.3765210>.
- Lei, Chong, Belaton, Bahari, 2025. Non-control-data attacks and defenses: a review. *IEEE Access*. 13, 172507–172519. <https://doi.org/10.1109/ACCESS.2025.3617245>.
- Liu, Zian, Liu, Shigang, Pan, Lei, Chen, Chao, Ahmed, Ejaz, Zhang, Jun, Liu, Dongxi, 2025. VulMatch: binary-level vulnerability detection through signature. In: Proceedings of the 18th International Conference on Network and System Security. Berlin/Heidelberg, Germany, 15564. Springer-Verlag, pp. 409–427. https://doi.org/10.1007/978-981-96-3531-3_20.
- Nagy, Stefan, Hicks, Matthew, 2019. Full-speed fuzzing: reducing fuzzing overhead through coverage-guided tracing. In: Proceedings of the IEEE Symposium on Security and Privacy (SP). San Francisco, USA. IEEE Computer Society, pp. 787–802. <https://doi.org/10.1109/SP.2019.00069>.
- National Institute of Standards and Technology, 2026. NIST updates NVD operations to address record CVE growth. Apr. 15. <https://www.nist.gov/news-events/news/2026/04/nist-updates-nvd-operations-address-record-cve-growth>. Accessed: 2026-08-01.
- NIST National Vulnerability Database, 2026. NVD dashboard: 2025 vulnerability volume statistics. Available. <https://nvd.nist.gov/general/visualizations/vulnerability-visualizations/cvss-severity-distribution-over-time>. Accessed: 2026-08-01.
- Rababah, Baha, Wu, Shang, Kwiatkowski, Matthew, Leung, Carson, 2024. Cuneyt Gurcan Akcora. SoK: prompt hacking of large language models. In: Proceedings of IEEE International Conference on Big Data (BigData). Washington, USA. IEEE Computer Society, pp. 5392–5401. <https://doi.org/10.1109/BigData62323.2024.10825103>.
- Radareorg, 2026. radare2. Available. <https://github.com/radareorg/radare2>. Accessed: 2026-08-01.
- SANS Institute, 2026. RSAC 2025: the SANS institute's top 5 most dangerous new attack techniques to watch, available. Accessed: <https://www.sans.org/press/announcements/rsac-2025-sans-top-5-most-dangerous-new-attack-techniques>. Accessed: 2026-08-01.
- Shao, Minghao, Jancheska, Sofija, Udeshi, Meet, Dolan-Gavitt, Brendan, Xi, Haoran, Milner, Kimberly, Chen, Boyuan, Yin, Max, Garg, Siddharth, Krishnamurthy, Prashanth, Khorrami, Farshad, Karri, Ramesh, Shafique, Muhammad, 2024. NYU CTF bench: a scalable open-source benchmark

- dataset for evaluating LLMs in offensive security. In: Proceedings of the 38th International Conference on Neural Information Processing Systems (NeurIPS'24). Vancouver, British Columbia, Canada, 37. Curran Associates Inc, pp. 57472–57498. <https://doi.org/10.48550/arXiv.2406.05590>.
- Shoshitaishvili, Yan, Wang, Ruoyu, Salls, Christopher, Stephens, Nick, Polino, Mario, Dutcher, Andrew, Grosen, John, Feng, Siji, Hauser, Christophe, Kruegel, Christopher, Vigna, Giovanni, 2016. SOK: (State of) the art of war: offensive techniques in binary analysis. In: Proceedings of the IEEE Symposium on Security and Privacy (S&P). San Jose, USA. IEEE Computer Society, pp. 138–157. <https://doi.org/10.1109/SP.2016.17>.
- Vishnyakov, A.V., Nurmukhametov, A.R., 2021. Survey of methods for automated code-reuse exploit generation. *Programm. Comput. Soft.* 47 (4), 271–297. <https://doi.org/10.1134/S0361768821040071>.
- Wang, Yan, Zhao, Zixuan, Liu, Bingchang, Zhang, Chao, Li, Wenjie, Chen, Kaixiang, Xiang, Xiaobo, Gong, Xiaorui, Revery, Wei Zou., 2018. From proof-of-concept to exploitable (one step towards automatic exploit generation). In: Proceedings of the ACM Conference on Computer and Communications Security. New York, USA. Association for Computing Machinery, pp. 1914–1927. <https://doi.org/10.1145/3243734.3243847>.
- Lingzhi Wang; Xinyi Shi; Ziyu Li; Yi Jiang; Shiyu Tan; Yuhao Jiang; Junjie Cheng; Wenyan Chen; Xiangmin Shen; Zhenyuan Li; Yan Chen. Automated penetration testing with LLM agents and classical planning. *arXiv Preprint arXiv:2512.11143*, 2025. [10.48550/arXiv.2512.11143](https://arxiv.org/abs/2512.11143).
- Wu, Yifan, Li, Yinshuai, Zhu, Hong, Zhang, Yinqian, 2024. SAEG: stateful automatic exploit generation. In: Proceedings of the 29th European Symposium on Research in Computer Security. Berlin/Heidelberg, Germany, 14915. Springer, pp. 127–145. https://doi.org/10.1007/978-3-031-70903-6_7.
- Xu, Shenglin, Wang, Yongjun, 2022. BofAEG: automated stack buffer overflow vulnerability detection and exploit generation based on symbolic execution and dynamic analysis. *Secur. Commun. Network.* 2022, 1–9. <https://doi.org/10.1155/2022/1251987>.
- Yang, Mingyue, Lie, David, Papernot, Nicolas, 2024. Exploring strategies for guiding symbolic analysis with machine learning prediction. In: Proceedings of the IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER). Rovaniemi, Finland. IEEE Computer Society, pp. 659–669. <https://doi.org/10.1109/SANER60148.2024.00073>.
- Yin, Jiao, Tang, MingJian, Cao, Jinli, Wang, Hua, 2020. Apply transfer learning to cybersecurity: predicting exploitability of vulnerabilities by description. *Knowl. Based Syst.* 210, 106529. <https://doi.org/10.1016/j.knosys.2020.106529>.
- Yin, Jiao, Tang, MingJian, Cao, Jinli, Wang, Hua, You, Mingshan, 2022. A real-time dynamic concept adaptive learning algorithm for exploitability prediction. *Neurocomputing* 472, 252–265. <https://doi.org/10.1016/j.neucom.2021.01.144>.
- Zeratool, 2026. Available. <https://github.com/ChrisTheCoolHut/Zeratool>. Accessed:-08-01.